

COMPUTER SCIENCE AND ENGINEERING
Indian Institute of Technology Palakkad
CS5024: Fundamentals of Computer Graphics
Assignment 2 & 3: Spacecraft Mission

1 Few Essential Rules

- Work in a team of two or three students. Continue with the same team for both assignments.
- At least one team member must upload the submission to Moodle.
- Discussion is encouraged, but the implementation and report must be your team's own work. Acknowledge all external references, assets, and libraries.
- Use version control where possible. Keep solution code and reports in a private repository; do not publish them online.
- Write readable, modular code and submit an organised project with clear build and execution instructions.

2 Assignment 2 Statement

Spacecraft Modelling, Rendering, and Launch

Choose a real spacecraft mission and recreate its launch in an interactive graphics application. Research the mission so that the geometry, colours, textures, launch infrastructure, and sequence of events resemble the actual mission. Your scene must include the launch vehicle, its payload, the launch site, Earth, the atmosphere, and the surrounding space.

A launch vehicle may have several rocket stages and a payload. The payload may be an Earth-orbiting satellite, a lunar spacecraft, or a deep-space probe. Select a mission whose important components you can represent convincingly.

Modelling Requirements

1. **Earth and its surroundings.** Build and texture Earth, represent its atmosphere, and render a space background. A skybox or an environment sphere is suitable for space; an atmospheric-scattering shader is an optional way to render the atmosphere. Include the Moon or other celestial bodies when the chosen mission requires them. Account for planetary rotation and distinguish motion under Earth's gravity and atmospheric resistance from motion in space, where atmospheric drag is absent.

2. **Earth viewer:** `a2-model-0`**.** Produce a separate executable that displays Earth, its atmosphere, and space. Use `A/a` and `D/d` to rotate Earth about its axis. Use the four arrow keys to translate the camera along the X and Y axes, and `[` and `]` for translation along Z. The camera must support views near the surface as well as views from space.

3. **Launch-site viewer:** `a2-model-1`**.** Model the launch platform with ground texture.

Continued on next page...

4. **Vehicle and payload viewer:** `a2-model-2`**.** Construct the launch vehicle and payload, including the stages and detachable components relevant to the mission. For example, your rocket may have three stages. Each stage detaches after reaching a certain height.

5. **Use of Blender.** You may model and texture the launch vehicle in Blender. You may use AI/LLM for this. Document which component, if any, was produced in Blender.

Scene Assembly and Launch Animation

1. **Integrated application:** `a2-scene`**.** Combine the models in a single scene. Position the launch site at the correct geographical location on Earth and arrange the vehicle and payload appropriately. You may determine translation, rotation, and scale interactively and then record the resulting transformations in the scene file.

2. **Lighting.** Illuminate the scene consistently with the time of day represented by the launch. Ensure that the vehicle, site, Earth, atmosphere, and exhaust remain visually coherent.

3. **Cameras.** Provide at least three cameras as described above: the *Earth viewer* camera will produce a distant view covering the complete launch, including the planets; the *Launch-site* camera will show spacecraft details and *Vehicle and payload* camera always move with the launch vehicle.

Continued on next page...

3 Assignment 3 Statement

Trajectory Design and Keyframe Animation

Extend the spacecraft scene from Assignment 2 into a complete mission animation. Finish any remaining models or textures, and add any planets, moons, or other objects needed for the mission. Plan separate paths for the launch vehicle, detachable stages or boosters, and payload, including the payload's final trajectory. Also plan the camera position and look-at target throughout the animation. Camera changes may be used to present different phases of the mission or show close-up views.

Interactive Bézier Trajectory Editor

1. Implement a mouse-based interface for placing control points in three-dimensional scene space. Allow the user to navigate with the arrow keys and make the position of every selected point visible, for example by drawing a small sphere.

2. Pressing `T/t` must save or append the current trajectory's control points to `trajectory.txt`. Store each trajectory as a block beginning with its number of control points, followed by one (x, y, z) coordinate triple per line:

   ```
   number_of_control_points
   x_1 y_1 z_1
   x_2 y_2 z_2
   ...
   x_n y_n z_n
   ```

   Store additional trajectories by appending further blocks of the same form.

3. Pressing `Y/y` must load the saved trajectories and display them together, using a different color for each curve. Create and store every trajectory required for the spacecraft components and camera motion.

Keyframe Representation and Playback

1. Create `keyframes.txt`. Each row represents one keyframe and must contain an activity flag and a curve-parameter value for every trajectory. An activity flag is either 0 (inactive) or 1 (active); the parameter identifies a position on the corresponding Bézier curve. For four trajectories, the trajectory fields have the form:

   ```
   active_1 u_1 active_2 u_2 active_3 u_3
   ```

   For example, the trajectories might describe the main rocket motion, detaching components' trajectories, the payload, and the camera eye. Include further trajectories or fields whenever the mission or camera requires them.

2. Generate intermediate frames by interpolating between successive keyframes. Continuous quantities, such as curve parameters and translations, can be interpolated continuously. Discrete flags must change at the appropriate event boundary and must not take fractional values. With a

Continued on next page...

fixed number of intermediate frames per keyframe interval, the difference between successive curve-parameter values controls progress along the path and hence the apparent speed.

3. Animate both the scene objects and the camera. Coordinate the camera eye and look-at target to keep relevant events visible, and support planned camera changes where useful. Make stage detachment, payload deployment, and articulated motion consistent with the mission sequence.

4. Pressing `P/p` must start the animation.

4 Bonus

Up to 20% above the base marks is available for substantial extensions related to these assignments. Possible extensions include interactive editing of existing control points, more convincing exhaust and atmospheric effects, or physically motivated motion of separated rocket stages. Clearly identify each extension in the report and explain how it goes beyond the required work. Completing a compulsory feature does not count as a bonus.

5 Submission Details

Submit a clean source-code archive to Moodle. **Submission deadline: 8th November**

Include an originality declaration and acknowledge all references and third-party resources. Ensure that the project builds and runs using the submitted instructions.